LAPORAN PRAKTIKUM LINKED LIST
MATA KULIAH STRUKTUR DATA

```java
package Praktikum9;

public class Node {
    private String data;
    private Node nextReference;

    // constructor
    public Node(String data) {
        this.data = data;
        this.nextReference = null;
    }

    // getter untuk mengambil nilai data
    public String getData() {
        return data;
    }

    // setter untuk mengubah nilai data
    public void setData(String data) {
        this.data = data;
    }

    // getter untuk mengambil nilai nextReference
    public Node getNextReference() {
        return nextReference;
    }

    // setter untuk mengubah nilai nextReference
    public void setNextReference(Node nextReference) {
        this.nextReference = nextReference;
    }
}
```

1. Penjelasan Program (Class Node)

Class Node merupakan class yang digunakan untuk mempresentasikan satu elemen (simpul) dalam struktur data **Linked List**. Setiap node menyimpan data dan referensi ke node berikutnya.

a. Atribut (Variable)
   - data
     Digunakan untuk menyimpan nilai bertipe String pada node.
   - nextReference
     Digunakan untuk menyimpan referensi atau alamat ke node berikutnya dalam linked list.
     Jika bernilai null, berarti node tersebut adalah node terakhir.

b. Constructor
   - Constructor ini digunakan untuk membuat objek node baru.
   - Parameter data akan langsung disimpan ke dalam atribut data.
   - nextReference di-set null karena saat node dibuat, belum terhubung ke node lain.

c. Method Getter dan Setter

   Getter untuk data
   - Mengambil nilai data dari node.

   Setter untuk data
   - Digunakan untuk mengubah nilai data pada node.

   Getter untuk nextReference
   - Menambil node berikutnya.

   Setter untuk nextReference
   - Digunakan untuk menghubungkan node ini dengan node lain.

2. Menganalisi Alus Program

   Class Node tidak berjalan sendiri karena bukan class utama class ini biasanya digunakan oleh class lain (SingleLinkedList) untuk:

   1. Untuk membuat node baru
   2. Menyimpan data
   3. Menghubungkan node satu dengan lainnya

```java
package Praktikum9;

public class SingleLinkedList {
    private Node head;
    private Node tail;
    private int size;

    public SingleLinkedList() {
        this.head = null;
        this.tail = null;
        this.size = 0;
    }

    public boolean isEmpty() {
        return (this.head == null) && (this.tail == null);
    }

    public int getSize() {
        return size;
    }

    public void addFirst(Node node) {
        if (!isEmpty()) {
            node.setNextReference(head);
            head = node;
        } else {
            node.setNextReference(null);
            tail = node;
            head = node;
        }
        ++this.size;
    }

    public void addLast(Node node) {
        if (!isEmpty()) {
            node.setNextReference(null);
            this.tail.setNextReference(node);
            tail = node;
        } else {
            node.setNextReference(null);
            tail = node;
            head = node;
        }
        ++this.size;
    }

    public void display() {
        Node pointer;
        pointer = head;
        System.out.println("Size : " + this.size);
        while (pointer != null) {
            System.out.println(pointer.getData());
            pointer = pointer.getNextReference();
        }
    }

    public Node search(String data) {
        Node pointer;
        if (isEmpty()) {
            return null;
        }
        pointer = head;
        while (pointer != null) {
            if (pointer.getData().contentEquals(data)) {
                return pointer;
            }
            pointer = pointer.getNextReference();
        }
        return null;
    }

    public void deleteHead() {
        if (!isEmpty()) {
            Node pointer;
            pointer = head;
            head = pointer.getNextReference();
            pointer = null;
            // Jika setelah dihapus list jadi kosong, tail juga harus null
            if (head == null) {
                tail = null;
            }
            --this.size;
        }
    }
}
```

1. Penjelasan Program (Class SingleLinkedList)

   Class SingleLinkedList digunakan untuk mengelola struktur data **single linked list**, yaitu kumpulan node yang saling terhubung satu arah.

   a. Atribut
      - head ➔ menunjuk node pertama dalam list
      - tail ➔ menunjuk node terakhir dalam list
      - size ➔ menyimpan jimlah node dalam list

   b. Constructor
      - Menginisialisasi list dalam keadaan kosong
      - head dan tail bernilai null
      - size = 0

   c. Method isEmpty()
      - Mengecek apakah linked list kosong
      - Mengembalikan nilai true jika kosong, false jika tidak

   d. Method getSize()
      - Mengembalikan jumlah node dalam list

   e. Method addFirts(Node node)

      Fungsi: menambahkan node di awal list

      Alur:
      - Jika list tidak kosong:
        - Node baru menunjuk ke head lama
        - Head dipindah ke node baru
      - Jika kosong
        - head dan tail sama-sama menunjuk node baru
      - Size bertambah

   f. Method addLast(Node node)

      Fungsi: menambah node di akhir list

      Alur:
      - Jika list tidak kosong
        - Node baru jadi next dari tail
        - Tail di pindahkan ke node baru
      - Jika kosong
        - Head dan tail menunjuk node baru

- Size bertembah

g. Method display()

Fungsi: menampilkan seluruh isi linked list

Alur:

2. Pointer dimulai dari head
3. Selama pointer tidak null:
    - Cetak data
    - Geser pointer

h. Method search(String data)

Fungsi: mencari node berdasarkan data

Alur:

- Jika kosong ➔ return null
- Loop dari head:
    - Jika data cocok ➔ return data
- Jika tidak ditemukan ➔ return null

i. Method deleteHead()

Fungsi: menghapus node pertama

Alur:

1. Simpan head ke pointer
2. Head digeser ke node berikutnya
3. Pointer dihapus (null)
4. Jika list kosong:
    - Tail juga di-set null
5. Size berkurang

2. Analisis Alur Program

Class ini bekerja Bersama class Node.

Alur kerja umum:

1. Membuat objek SingleLinkedList
2. Menambahkan node (addFirts / addLast)
3. Node saling terhubung melalui nextReference
4. Data bisa :
    - ditampilkan(display)
    - dicari(search)

- dihapus(deleteHead)

Struktur data yang terbentuk:

Head ➔ node1 ➔ node2 ➔ node3 ➔null

^

||

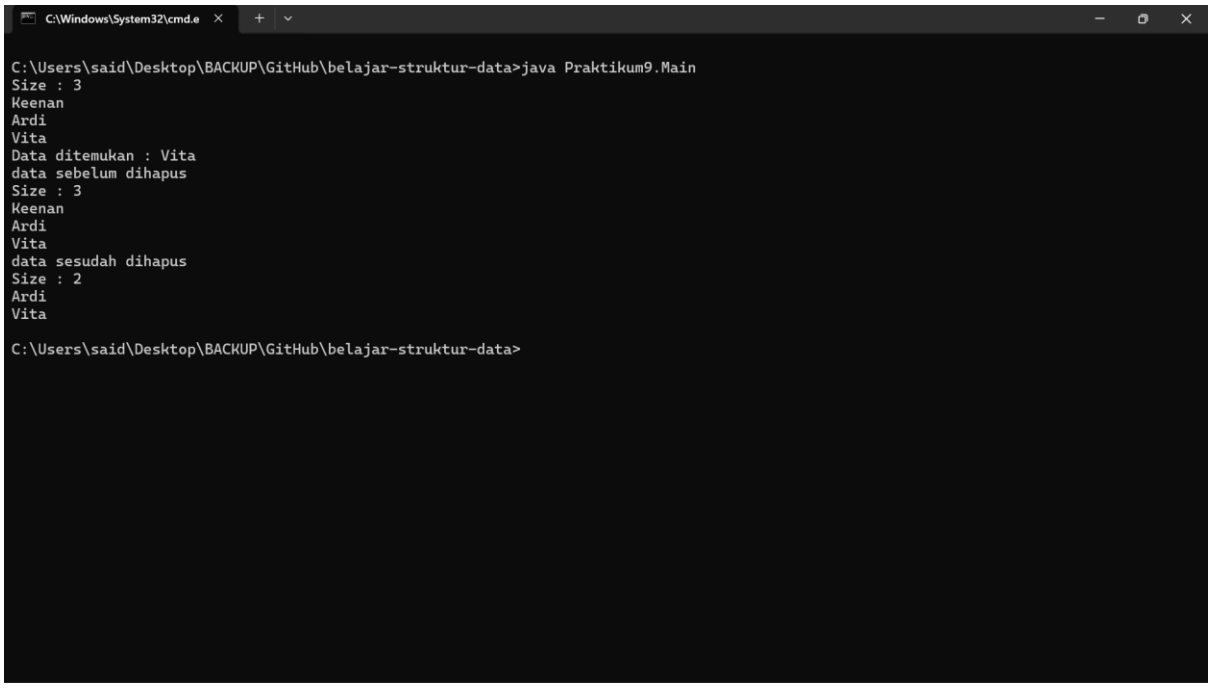
Tail

```java
package Praktikum9;

public class Main {
    public static void main(String[] args) {
        // LinkedList ll = new LinkedList();
        SingleLinkedList sll = new SingleLinkedList();
        Node data1 = new Node("Ardi");
        Node data3 = new Node("Keenan");
        Node data2 = new Node("Vita");
        sll.addLast(data1);
        sll.addLast(data2);
        sll.addFirst(data3);
        sll.display();

        Node hasilCari;
        hasilCari = sll.search("Vita");
        if (hasilCari == null) {
            System.out.println("Data tidak ditemukan");
        } else {
            System.out.println("Data ditemukan : " + hasilCari.getData());
        }
        System.out.println("data sebelum dihapus");
        sll.display();
        sll.deleteHead();
        System.out.println("data sesudah dihapus");
        sll.display();
    }
}
```

1. Penjelasan Program (Class Main)
   Class Main merupakan class utama yang berfungsi untuk menjalankan program dan menguji semua methode pada SingleLinkedList.
   a. Pembuatan objek
      - Membuat objek linked list kosong
   b. Pembuatan node
      - Membuat 3 node dengan data:
        - Ardi
        - Keenan
        - Vita

c. Menambah Data ke Linked List
   Alur:
   1. Ardi ➔ ditambahkan di akhir (jadi head & tail)
   2. Vita ➔ ditambahkan di akhir
   3. Keenan ➔ ditambahkan di awal

   Struktur akhir
   Keenan ➔ Ardi ➔ Vita ➔ null
d. Menampilkan Data
   Menampilkan seluruh isi linked list.
e. Pencarian Data
   - Mencari data “Vita”
   - Jika ditemukan ➔ tampilkan
   - Jika tidak ➔ tampilkan pesan tidak ditemukan
f. Menampilkan Data Sebelum dan Sesudah Hapus
   - Menampilkan data sebelum penghapusan
   - Menghapus node pertama (Keenan)
   - Menampikan data setelah penghapusan



2. Analisis Alur Program Keseluruhan:
   Program berjalan dengan urutan:
   1. Membuat linked list kosong
   2. Membuat beberapa node
   3. Menambahkan node ke list (awal & akhir)
   4. Menampilkan isi list
   5. Mencari data tertentu
   6. Menampilkan data sebelum dihapus
   7. Menghapus node pertama
   8. Menampilkan data setelah di hapus